

字符串基础

2026 河南省信息学竞赛暑期训练营

Displace

香港科技大学（广州）

xwang161@connect.hkust-gz.edu.cn

2026.7.4



个人介绍

- 毕业于焦作市第一中学，现就读于香港科技大学（广州），任香港科技大学（广州）ACM 一队队长。
- NOI2023 银牌
- ICPC 区域赛 3 个金牌，CCPC 区域赛 3 个金牌
- 参与过多个校级赛事命题
- 擅长领域：字符串，数据结构
- 薄绿厨

今日内容概览

- Hash
- KMP
- Z-func
- ACAM
- Manacher
- 杂题选讲

- 字符串可以视作一个大进制数（比如小写字母串为 26 进制数）。
- 因此，我们试图找到一个映射，使得我们可以用整数的方式判断字符串的相等性。（注意，不同于整数，比较大小是相对困难的。）
- 我们先尝试找一个模数 mod 和一个进制 $base$ ，设计一个哈希函数：
-

$$f(S) = \begin{cases} 0, & S = \emptyset \\ f(S[:|S| - 1]) \times base + S[|S|] \mod mod & \text{otherwise} \end{cases}$$

- 很显然是有概率产生哈希冲突的，冲突的概率与模数的大小密切相关，方便起见我们常用 `int` 范围内的大质数。

- 然而，这种哈希方式有不可规避的缺陷：存在确定性构造，使得无论选取怎么样的模数和进制，都会产生哈希冲突。
- 所以如果需要使用字符串哈希，特别是要区分子串的哈希值，我们通常要找两套哈希函数组成 pair，两个字符串的相等条件变为两个哈希值都相同。
- 根据本人经验，
 $mod_1 = 998244353, base_1 = 137, mod_2 = 993244853, base_2 = 13331$
是一个良好的哈希方式。

- 我们不满足于只能求某个串的哈希值，我们希望能够求出这个串任意子串的哈希值。
- 简单推导就可以得到：
- $f(S[l:r]) = f(S[:r]) - f(S[:l-1]) \times base^{r-l+1} \mod mod。$
- 线性预处理即可获得串 S 任意子串的哈希值。

- 哈希的常见应用:
- 替代 KMP
- 二分求 LCP
- 判回文串
- 去重
-

双串模式匹配

问题描述

给定一个模式串 S 和一个文本串 T ，查询这个模式串是否在文本串里出现过。

朴素的做复杂度为 $O(n^2)$ ，KMP 是一个复杂度为 $O(n)$ 的高效解决次问题的算法。

border 的定义

定义字符串 S 的 border 为：非平凡前缀 $S[:i]$ ，满足
 $S[:i] = S[|S| - i + 1 :]$ 。

KMP 求出了模式串每个前缀的最长 border，辅助解决双串匹配问题。

算法流程

记失配函数 $next[i]$ ，表示最长的前缀为 $S[: next[i]]$ ，使得

$S[: next[i]] = S[i - next[i] + 1 : i]$ 。

考虑暴力求失配函数，容易发现 $S[: next[next[i]]]$ 仍然是 $S[: i]$ 的 border，我们可以得到以下的算法：

- 记 $next[1] = 0$ ，从 $i = 2$ 开始增量求失配函数，同步记录 $j = 1$ 。
- 只要 $S[j + 1] \neq S[i]$ 并且 $j \neq 0$ ， $j \leftarrow next[j]$ 。
- 如果 $S[j + 1] = S[i]$ ， $j \leftarrow j + 1$ 。
- $next[i] \leftarrow j$ 。

j 每轮最多增加 1，时间复杂度为 $O(n)$ 。

题意

给出 n 个字符串 S , 对于每个字符串求出它每个前缀的满足长度不超过 $\frac{|S|}{2}$ 的 border 个数 (即前后缀不重叠)

$1 \leq n \leq 5, 1 \leq |S| \leq 10^6$

- 考虑 border 的 border 还是 border 。求出失配函数后，将 i 连向 $\text{next}[i]$ 。
- 容易发现这是一棵树。
- 我们记录一下每个点到根的距离，答案就是 i 不超过一半的最长 border 到根的距离，可以做到 $O(|S|)$ 。
- 事实上，这棵树就是 fail 树。

题意

给出一个字符串 s , m 次询问, 求 $s[:p]$ 和 $s[:q]$ 的最长公共 Border 长度。

$$|s| \leq 10^6, m \leq 10^5$$

- fail 树的父亲指向的是节点代表串的最长 Border，所以建出 fail 树后问题转化为两点 lca。
- 复杂度为求 lca 复杂度，取决于你想怎么求 lca。
- fail 树的性质极为重要，正是这棵树使得很多复杂的字符串问题可以通过树形数据结构解决。

题意

定义函数 F_G 为满足下列要求的正整数 x 的数量：

- $x \leq |G|$
- $G[:x] = G[|G| - x + 1:]$
- $[:x]$ 与 $[|G| - x + 1:]$ 的交集非空且大小是 k 的倍数, k 为固定参数。

给出一个字符串 S , 求 $\forall i, F_{S[:i]}$ 。

$|S| \leq 10^6$

- 花里胡哨，实际上就是第一题的拓展。
- 这里需要多维护一个桶求满足第三个条件的 fail 树祖先数量。
- 直接 dfs fail 树即可，让桶里在任意时刻都仅包含从根到当前点这条链上的信息。
- 复杂度为 $O(|S|)$ 。

题意

给出一个 01 串 S ，求出长度为 n 的 01 串 T 的数量，使得 T 无限循环拼接后含有 S 作为子串。

$|S| \leq n \leq 40$ 。

- 考虑极限，假设 T^∞ 匹配到了 $|S|$ 的某个位置 p ，那么在后面再匹配一个 T ，必须还匹配在位置 p 。
- 并且，如果 T^∞ 没有完整匹配过 $|S|$ ，那么在后面再匹配一个 T ，仍然不会完整匹配过 $|S|$ 。
- 于是可以枚举 p 做 DP，求出所有不含 S 作为子串的 T ，再求补集即可。
- 复杂度为 $O(n|S|^2)$ 。

题意

对于字符串 s ，定义 z_i 表示 $s[i:]$ 和 s 的最长公共前缀长度。
用在线的方式给出一个字符串，给出当前的 s 后，求出

$$\sum_{i=1}^{|s|} \sum_{j=i}^{i+z_i-1} A_j \times B_i。$$

$$|s| \leq 3 \times 10^5。$$

- 将 $\sum_{i=1}^{|s|} \sum_{j=i}^{i+z_i-1} A_j \times B_i$ 拆分到 A 上, 我们将问题转化为寻找每个 A 与哪些 B 相乘。
- 类比扫描线, 相当于每个 B_i 在 A 上留下了一个区间 $[i, i + z_i - 1]$, 它们的起点互不相同。
- 容易发现与 A_j 相乘的 $\sum B_i$, 可以由 A_{j-1} 的 $\sum B_i$ 减去若干个, 再最多加上一个得到。

- 考虑增量，如果一个 z_i 在这回合不会增加，在之后的回合也不会增加。
- 只需要快速找出会增加的 z_i 对应的下标。
- 容易发现这些下标是 $i-1$ 在 fail 树上的父亲，如果暴力跳肯定复杂度不对。
- 不过如果我们把后一个字符相同的祖先缩起来，那么复杂度就是均摊线性的，这个可以用类似并查集的思想实现。
- 复杂度为 $O(|s|)$ 。
- 实际上，这是一个用 KMP 实现的在线 Z 函数。

加强的双串模式匹配

问题描述

给出一个长度为 n 的字符串 s ，求出每个后缀和这个串本身的最长公共前缀 (LCP) 的长度。

例： $z(\text{"aaabaab"}) = [7, 2, 1, 0, 2, 1, 0]$ 。

定义函数 $z[i]$ 表示 s 和 $s[i:]$ 的 LCP 长度。

求解 z 函数的算法称为 Z Algorithm，也叫扩展 KMP (但实际和 KMP 没什么联系)。

算法流程

在该算法中，我们从 2 到 n 顺次计算 $z[i]$ 的值 ($z[0] = 0, z[1] = n$)。我们利用已经计算好的 $z[1], \dots, z[i-1]$ 来递推出 $z[i]$ 。

对于 i ，我们称区间 $[i, i + z[i] - 1]$ 是 i 的匹配段，也叫做 Z-box。

算法的过程中我们维护右端点最靠右的匹配段，记作 $[l, r]$ 。根据定义， $s[l:r]$ 是 s 的前缀。在计算 $z[i]$ 时我们保证 $l \leq i$ 。初始时 $l = r = 0$ 。

在计算 $z[i]$ 的过程中：

- 如果 $i \leq r$ ，根据定义有 $s[i:r] = s[i-l+1:r-l+1]$ ，因此 $z[i] \geq \min(z[i-l+1], r-i+1)$ 。此时：
 - 若 $z[i-l+1] < r-i+1$ ，则 $z[i] = z[i-l+1]$ 。
 - 否则 $z[i-l+1] \geq r-i+1$ ，这是我们令 $z[i] = r-i+1$ ，然后暴力枚举下一个字符扩展 $z[i]$ ，直到不能扩展为止。
- 如果 $i > r$ ，直接暴力求出 $z[i]$ 。
- 在求出 $z[i]$ 后，如果 $i + z[i] - 1 > r$ ，则更新 $l = i, r = i + z[i] - 1$ 。

代码

```
inline void Z(char *s, int n) {  
    for (int i = 1; i <= n; i++) z[i] = 0;  
    z[1] = n;  
    for (int i = 2, l = 0, r = 0; i <= n; i++) {  
        if (i <= r) z[i] = min(z[i-l+1], r - i + 1);  
        while (i + z[i] <= n && s[i+z[i]] == s[z[i]+1]) ++z[i];  
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;  
    }  
}
```


复杂度分析

空间复杂度显然是 $O(n)$ 。

对于内层循环，每次执行都会使得 r 向后至少移 1 位，而 $r < n - 1$ ，所以总共只会执行 n 次。

对于外层循环，只有一遍线性遍历。

所以总时间复杂度为 $O(n)$ 。

两个串的 p 函数

定义两个串 s 关于 t 的 p 函数为: $p[i] = t$ 与 $[i:]$ 的 LCP 长度。

我们先求出 t 的 z 函数, 再递推 p 函数:

- 如果 $i \leq r$, 根据定义有 $s[i:r] = t[i-l+1:r-l+1]$, 因此 $p[i] \geq \min(z[i-l+1], r-i+1)$ 。此时:
 - 若 $z[i-l+1] < r-i+1$, 则 $p[i] = z[i-l+1]$ 。
 - 否则 $z[i-l+1] \geq r-i+1$, 这是我们令 $p[i] = r-i+1$, 然后暴力枚举下一个字符扩展 $z[i]$, 直到不能扩展为止。
- 如果 $i > r$, 直接暴力求出 $p[i]$ 。
- 在求出 $p[i]$ 后, 如果 $i + p[i] - 1 > r$, 则更新 $l = i, r = i + p[i] - 1$ 。

代码

```
inline void exkmp(char *s, int n, char *t, int m) {  
    Z(t, m);  
    for (int i = 1; i <= n; i++) p[i] = 0;  
    for (int i = 1, l = 0, r = 0; i <= n; i++) {  
        if (i <= r) p[i] = min(z[i-l+1], r - i + 1);  
        while (i + p[i] <= n && s[i+p[i]] == t[p[i]+1]) ++p[i];  
        if (i + p[i] - 1 > r) l = i, r = i + p[i] - 1;  
    }  
}
```

题意

给出一个字符串 S 。定义一个合法的拆分为 $S = (AB)^k C$, $k \geq 1$, 三个字符串都非空, 并且 $F(A) \leq F(C)$, $F(s)$ 表示字符串 s 中出现奇数次的字符数量。

两个方案被认为不同当且仅当 A, B, C 中至少有一个字符串不同, 求合法的拆分的方案数。

$|S| \leq 2^{20}$ 。

- 先忽略 $F(A) \leq F(C)$ 的限制。
- 不妨枚举 $D = AB$ 的长度。当 D 长为 x 时，不难发现 k 的取值范围是 $[1, \lfloor \frac{z[x+1]}{x} \rfloor]$ 。
- 用 exkmp 算法即可解决这个弱化问题。

- 再加上 $F(A) \leq F(C)$ 的限制。记 $f(l, r)$ 表示 $s[l, r]$ 中出现奇数次的字符的个数。
- 当 k 为奇数时，容易发现都和 $k = 1$ 的情况相同，于是只需要计算 $f(1, j) \leq f(x + 1, n), j \leq x - 1$ 的 j 的数量，与 $\lceil \frac{\lfloor \frac{z[x+1]}{x} \rfloor}{2} \rceil$ 相乘累加进答案。
- 当 k 为偶数时，容易发现都和 $k = 2$ 的情况相同，于是只需要计算 $f(1, j) \leq f(2x + 1, n), j \leq 2x - 1$ 的 j 的数量，与 $\lfloor \frac{\lfloor \frac{z[x+1]}{x} \rfloor}{2} \rfloor$ 相乘累加进答案。

[NOIP2020] 字符串匹配

- 现在只需要能支持快速查询 $f(1, x)$ 和 $f(x, n)$ ，后者可以预处理，前者要查询小于等于某个值的数量。
- 因为 $f(1, x)$ 的取值只有 26 种，可以直接用树状数组维护，足够通过。
- 时间复杂度为 $O(|s| \log 26)$ 。

题意

给出一个字符串 s ，定义完美前缀为：这个前缀同时是 s 的后缀。
求出完美前缀的个数和分别在 s 中的出现次数。
 $|s| \leq 10^5$ 。

- 实际上就是求每个 border 的出现次数。
- 构造字符串 $t = s + '|' + s$, 求出 t 的 z 函数。
- 求前缀 $s[:i]$ 的出现次数即为求后半部分的 z 函数中有多少个位置满足 $z[|s| + 1 + x] \geq x$, 开个桶求后缀和即可。
- 时间复杂度为 $O(|s|)$ 。

多串模式匹配

问题描述

给定 n 个模式串 S_i 和一个文本串 T ，查询每个模式串是否在文本串里出现过。

双串模式匹配我们可以使用 KMP 算法解决，但 KMP 算法不能很好地解决多串模式匹配。

定义

Trie 树可以将若干个字符串储存为树形结构，任意两个串的公共前缀共享同一条路径。

概述

AC 自动机是以 Trie 的结构为基础，结合 KMP 的思想建立的自动机。AC 自动机本质上是 Trie 上的自动机，可以接受给出的模式串集合中的串。

AC 自动机：失配指针

定义

状态 u 的 fail 指针指向另一个状态 v ，满足 v 代表的串是 u 的最长后缀。

与 KMP 的比较

- 共同点：两者同样是在失配的时候用于跳转的指针。
- 不同点：next 指针求的是最长 Border（即最长的相同前后缀），而 fail 指针指向所有模式串的前缀中匹配当前状态的最长后缀。

因为 KMP 只对一个模式串做匹配，而 AC 自动机要对多个模式串做匹配。有可能 fail 指针指向的结点对应着另一个模式串，两者前缀不同。

构建指针

参考 KMP 中构造 next 指针的思想。

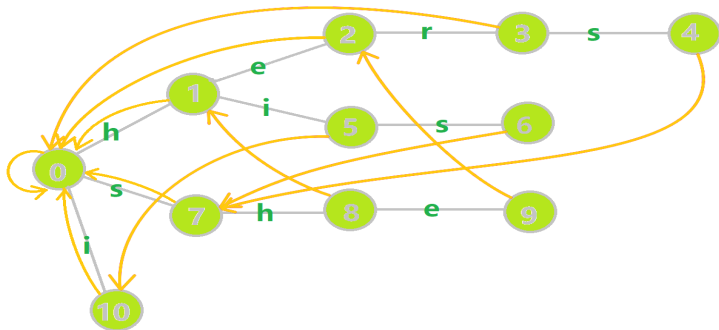
考虑字典树种当前的节点 u ， u 的父节点是 p ， p 通过字符 c 的边指向 u ，即 $\text{trie}(p, c)=u$ 。假设深度小于 u 的所有节点的 fail 指针都已求得。

- 如果 $\text{trie}(\text{fail}(p), c)$ 存在：则让 u 的 fail 指针指向 $\text{trie}(\text{fail}(p), c)$ 。相当于在 p 和 $\text{fail}(p)$ 后面加一个字符 c ，分别对应 u 和 $\text{fail}(u)$ ；
- 如果 $\text{trie}(\text{fail}(p), c)$ 不存在：那么我们继续找到 $\text{trie}(\text{fail}(\text{fail}(p)), c)$ 。重复判断过程，一直跳到 fail 指针直到根节点；
- 如果依然不存在，就让 fail 指针指向根节点。

AC 自动机：失配指针

图例

字符串集: {"i", "she", "his", "hers"}



概述

Trie 图基于 AC 自动机，把每个状态的每个字符出边都给出指向的状态，相当于 fail 指针的拓展，实用性更强。

相比于 AC 自动机，我们更经常使用 Trie 图。现在提到 AC 自动机时，我们一般默认指 Trie 图。

代码

```
1 void build() {
2     queue<int> q;
3     for (int i = 0; i < 26; i++)
4         if (tr[0].son[i]) q.push(tr[0].son[i]);
5     while (!q.empty()) {
6         int u = q.front();
7         q.pop();
8         for (int i = 0; i < 26; i++) {
9             if (tr[u].son[i]) {
10                 tr[tr[u].son[i]].fail = tr[tr[u].fail].son[i];
11                 q.push(tr[u].son[i]);
12             } else
13                 tr[u].son[i] = tr[tr[u].fail].son[i];
14         }
15     }
16 }
```

代码解释

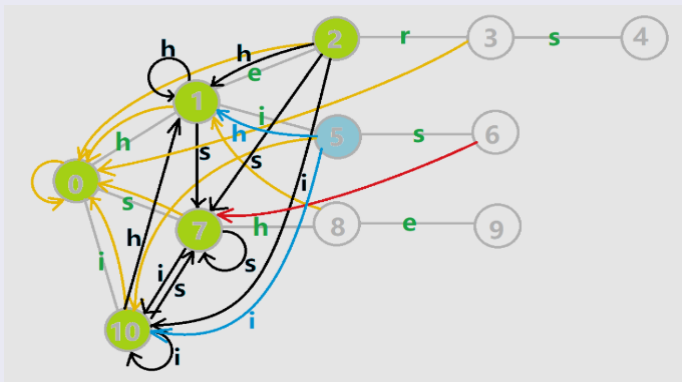
该函数将结点按 BFS 顺序入队，依次求 fail 指针。

每次取出队首的节点 u ， $\text{fail}(u)$ 在之前的过程中已求得，然后遍历出边：

- 如果 $\text{trans}(u, c)$ 存在，我们就讲 $\text{trans}(u, c)$ 的 fail 指针赋值为 $\text{trans}(\text{fail}(u), c)$ 。
- 否则，令 $\text{trans}(u, c)$ 指向 $\text{trans}(\text{fail}(u), c)$ 的状态。

代码解释

字符串集: {"i", "she", "his", "hers"}



多串模式匹配

- 如果使用 AC 自动机解决多串模式匹配，我们就需要像 KMP 一样，如果失配就不断跳 fail 指针，直到匹配或跳到根结束。
- 如果一个状态能被匹配，那么这个状态的所有 Border 也能被匹配，所以我们匹配到某个状态后，也应该不断跳 fail 指针标记所有被匹配的 Border。
- 利用均摊标记，我们可以在 $O(|T| + |\Sigma| \sum |S_i|)$ 的时间复杂度内解决这个问题， $|\Sigma|$ 表示字符集大小。
- 事实上，我们可以利用哈希表优化枚举出边的过程，复杂度可以优化到 $O(|T| + \sum |S_i|)$ 。

多串模式匹配

- 使用 Trie 图可以简化匹配的过程，直接走处理好的出边即可，不过之后均摊标记的步骤不能省略。
- 所以说 Trie 图相对于 AC 自动机，功能性没有太大优势，只是在实现难易度上具有优势。
- Trie 图的构建不好避免枚举字符集，一般认为复杂度为 $O(|T| + |\Sigma| \sum |S_i|)$ 。

题意

给出文本串 S 和 n 个模式串 T_i ，对每个模式串，求在 S 中出现的次数。
 $|S| \leq 2 \times 10^6, \sum T_i \leq 2 \times 10^5$

- 建出模式串的 AC 自动机，之后把 S 放在自动机上跑，每扩展一位就把所在节点标记一次，表示当下前缀的某个后缀匹配到了这个节点。
- 之后是统计答案，对于每个前缀的最长匹配后缀，其所有 border 也是被匹配的。
- 于是对于每个节点，其跳 fail 指针能到达的所有节点的标记需要累加上它的标记。

- 我们考虑 fail 指针形成的结构：每个节点指向一个深度比它浅的节点，根节点指向自己。
- 于是 fail 指针形成的结构是一棵树，在后文中我们称其为 fail 树。
- 于是一个节点的标记就是子树内标记和，利用 dfs 就可以统计答案了。
- 复杂度 $O(|S| + \sum |T_i|)$ 。

题意

给出 n 个 01 串 s_i , 求是否存在长度无限且不含任意一个 s_i 为子串的 01 串。

$$\sum |s_i| \leq 3 \times 10^4$$

- 建出 AC 自动机，考虑长度无限的字符串的等价定义。
- 实际上，我们只需要在 AC 自动机（这里指 Trie 图）上找到一个环，使得这个环不经过任何结束节点。
- 于是按照之前的方法把结束节点预处理出来，在剩下的图上找个环即可。
- 复杂度为 $O(\sum |s_i|)$ 。

题意

给出一个字符串 s , m 次询问, 每次询问一个字符串 t , 求其最长的前缀, 使得这个前缀是 s 的子串。

$$|s|, \sum |t| \leq 10^7$$

- 建出 AC 自动机，将 s 放在 AC 自动机上匹配，标记匹配节点。
- 之后根据 fail 树的性质，将被标记节点的 fail 树祖先标记。
- 之后只需要查询每个串的代表节点 Trie 树上最深的被标记祖先节点即可。
- 复杂度 $O(s + \sum |t|)$

- 如果使用 SAM（后缀自动机），本题的解法更为简洁：直接把 t 在 s 的 SAM 上做匹配，第一个失配的位置前面就是答案。

题意

有一台打字机，有以下三种功能：

- 输入一个小写英文字母；
- 输入'B'，删除最后一个输入的字符；
- 输入'P'，打印当前打字机存储的字符串。

给出长度为 n 操作序列，给出 m 个询问，求第 x 个打印的字符串在第 y 个打印的字符串里出现了多少次。

$$n, m \leq 10^5$$

[NOI2011] 阿狸的打字机

- 可以利用类似可持久化的思想建出来 Trie，具体的，遇到删除操作时将当前节点跳到父亲。之后再建 AC 自动机。
- 如果暴力求答案，考虑把第 y 个字符串涉及到的节点打上标记，之后查询第 x 个字符串的终止节点 fail 子树内有多少个标记点即可。
- 于是可以离线询问，扩展字符则给对应节点加标记，删除则删除当前标记并回退，查询时用一个树状数组统计子树答案即可。
- 复杂度为 $O((n + m) \log n)$ 。

最长回文子串

问题描述

给定一个字符串 s ，求出最长的 s 的子串，使得这个子串和其反串相同（即回文串）。

概念

Manacher 算法是一种用于求解单串回文结构的算法。

具体的，对于 s 中的每个位置 i ，Manacher 算法可以求出以位置 i 为中心的最长回文串的半径长度。

得到这个信息，对于处理回文串问题有非常大帮助。

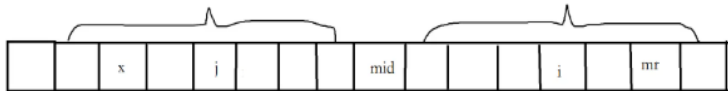
算法流程

为了减少讨论，我们通常先在 s 的开头和结尾各补一个不同的罕见字符。先考虑计算所有奇数长度回文子串。记 $p[i]$ 表示以位置 i 为中心的最长回文半径长度，我们仍然考虑用 $p[1], \dots, p[i-1]$ 递推 $p[i]$ 。

定义 mr 表示之前的 $i + p[i] - 1$ 的最大值（即最靠右的回文串）。同时记录这个回文中心为 mid （如果有多个，记录最靠右的）。

考虑当前位置 i ：

- 如果 $i \leq mr$ ，则设置初值为 $p[i] = \min(p[mid * 2 - i], mr - i + 1)$ 。
- 否则设置初值为 $p[i] = 1$ 。
- 之后暴力向两侧扩展 $p[i]$ ，并更新 mid 和 mr 。



复杂度分析

空间复杂度显然为 $O(|s|)$ 。

算法复杂度瓶颈在于暴力扩展部分的复杂度。

对于 $i \leq mr$ 的情况，当 $p[mid * 2 - i] < mr - i + 1$ 时，不会做暴力扩展。

注意到每次做暴力扩展时，都会让 mr 至少增加 1。

整个算法运行过程中， mr 不减，且上界为 $|s|$ 。

所以 Manacher 算法的时间复杂度为 $O(|s|)$ 。

完整算法

上述算法只能处理奇数长度回文子串，如何处理偶数长度？

将 s 的相邻两个字符之间添一个相同的罕见字符即可，注意这里的 s 是上述算法特殊处理过的 s 。

需要注意现在处理出来的回文半径是原来的 2 倍 ± 1 ，要根据奇偶性讨论一下。

Manacher 算法

代码

```
void build() {
    scanf("%s", c + 1), n = strlen(c + 1), s[++ cnt] = '~', s[++ cnt] = '#';
    rep(i, 1, n) s[++ cnt] = c[i], s[++ cnt] = '#';
    s[++ cnt] = '!';
}

void solve() {
    rep(i, 2, cnt - 1) {
        if(i <= mr) p[i] = min(p[mid * 2 - i], mr - i + 1);
        else p[i] = 1;
        while(s[i - p[i]] == s[i + p[i]]) ++ p[i];
        if(i + p[i] > mr) mr = i + p[i] - 1, mid = i;
        Ans = max(Ans, p[i]);
    }
    printf("%d", Ans - 1);
}
```

题意

定义翻转的操作：把一个串以最后一个字符作对称轴进行翻转复制。
求出所有可能的 S 的前缀，使得 S 可以由这个前缀翻转若干次得到。
 $|S| \leq 10^5$ 。

- 只翻转一次得到 S , 那么 S 的结构一定形如 $X + Y + rev(Y)$ 或 $X + Y + a + rev(Y)$, $X + Y$ 和 $X + Y + a$ 是分别的解。
- 用 manacher 求出回文半径后可以很容易判断这种情况。

- 翻转两次以上，最后一次之前的翻转都是完整的。
- 于是可以递推，先把上页说的一次翻转的位置标记。如果一个位置 i 加上回文半径 $p[i]$ 后能到达一个标记的位置，那么也标记当且位置。
- 复杂度是线性的。

[国家集训队] 最长双回文串

题意

给出一个字符串 S 。定义 s 双回文串当且仅当 s 可以被拆为 $x + y$ 并且 x 和 y 都是回文串。

求出 S 的最长双回文子串的长度。

$|S| \leq 10^5$ 。

[国家集训队] 最长双回文串

- 我们可以处理出每个位置 i : 以 i 开头的最长回文子串 $l[i]$ 和以 i 结尾的最长回文子串 $r[i]$ 。答案就是 $r[i] + l[i + 1]$ 的最大值。
- 用 Manacher 求出最长回文半径, 我们可以先给最长的回文串 (即 $[i - p[i] + 1, i + p[i] - 1]$) 打上标记, 之后可以向内每次减少 2 递推。
- 这个可以利用类似差分的思想完成, 会有一些奇偶性的细节, 留作练习。
- 时间复杂度 $O(|S|)$ 。

尝试上点强度！



题意

给定一个长为 N 的整数序列 A ，满足 $0 \leq A_i \leq \min\{i-1, N-i\}$ 。
构造字典序最小的正整数序列 S ，满足下面的条件，或者报告无解：

- $\forall 1 \leq i \leq n$ ，序列 $S_{i-A_i}, S_{i-A_i+1}, \dots, S_{i+A_i}$ 为回文串；
- $\forall 1 \leq i \leq n$ ，若 $2 \leq i-A_i \leq i+A_i \leq n-1$ ，则序列 $S_{i-A_i-1}, S_{i-A_i}, \dots, S_{i+A_i+1}$ 为非回文串

$N \leq 2 \times 10^5$ 。

[ABC349G] Palindrome Construction

- 把 Manacher 的过程逆过来，我们依次决定字符之间的相等/不等关系。
- 如果 $i \leq mr$ 并且 $p[mid * 2 - i] < mr - i + 1$ ，则此处没法向外扩展，必须要 $p[mid * 2 - i] = a[i]$ ，否则无解。
- 否则就向外扩展，扩展的同时用并查集维护字符之间的相等关系。
- 最后把两端点的不等关系再单独维护一下。
- 过程中只要出现了 $p[i] > a[i]$ ，就无解。
- Manacher 运行结束后，先把不等关系都拿出来，如果出现了在同一集合，则无解；否则贪心构造即可。
- 时间复杂度 $O(N)$ 。

题意

给出两个字符串集合 $\{pre_n\}$ 和 $\{suf_m\}$ ，再给出一个字符串 s ，求出它符合所有下面要求的子串数量。

- 存在一个前缀属于 $\{pre_n\}$;
- 存在一个后缀属于 $\{suf_n\}$;

$$\sum |pre_i| + \sum |suf_i| \leq 10^6。$$

[HDU8034] 双端魔咒

- 定义 $u[i]$ 为满足 $s[i : i + len - 1]$ 是属于 $\{pre_n\}$ 中最小的 len , 类似的对 $\{suf_m\}$ 定义 $v[i]$ 。
- 题意即求 $i + u[i] - 1 \leq j, i \leq j - v[j] + 1, i < j$ 的 (i, j) 对数量。
- 如果可以预处理出 $u[i], v[i]$, 那么剩下的只需要做数点问题。

- 将 $\{suf_m\}$ 和翻转的 $\{pre_n\}$ 分别建 AC 自动机，自动机上求出每个点最浅的是完整串的祖先。
- 在 $\{suf_m\}$ 的自动机上正序匹配，即可求出 $v[i]$ 。在翻转的 $\{pre_n\}$ 的自动机上倒序匹配，即可求出 $u[i]$ 。
- 这部分复杂度是线性的，复杂度瓶颈在于上页的数点问题。

题意

给出两个字符串集合 $\{s_n\}$ 和 $\{t_m\}$, 判断是否存在一个字符串 S , 使得存在 $1 \leq i < j \leq n$, s_i 和 s_j 都在 S 中出现, 且 $\{t_m\}$ 任何一个串都不在 S 中出现。

$$\sum |s_i| + \sum |t_i| \leq 10^6。$$

- 把两个字符串集合放一起建 Trie 图，将包含 t_i 的节点删除，这个可以通过在 trie 图上打标记实现。
- 接下来只需要找一条路径，包含两个不同 s_i 。
- 将删点后的 Trie 图缩点。如果一个 scc 有两个不同 s_i ，可以直接结束了；否则在缩点后的 DAG 上做 DP。
- 本题比较考验实现，大体思想是记录自身代表的字符串数量和通过有向边延伸过来的字符串数量，如果只有一个就额外记录编号辅助去重。

题意

m 次操作，每次给出一个字符串 s_i ，在一个字符串多重集合上三种操作：

- 加入这字符串到集合中。
- 把这个字符串从删除。
- 查询这个集合每个串在当前串中出现的次数之和。

$$\sum |s_i| \leq 3 \times 10^5。$$

- 众所周知，AC 自动机暂时没有在线增量构造算法。这里要介绍一个不可在线结构强行加 $\log$ 在线化的 trick。
- 假设我们要维护某种数据结构，需要支持在线插入一个新元素，查询某些信息。我们将这些元素分组，具体的，以总元素数目的二进制拆分为标准分组。
- 进行修改时，我们会将连续的低位合并，这时我们进行暴力合并。
- 将复杂度分摊到每个元素身上，不难发现，每个元素所处的组的编号是只会增加的，所以每个元素最多被暴力合并 $\lceil \log_2 n \rceil$ 次。
- 于是这个做法的合并复杂度为 $O(n \log n)$ ，通常时间复杂度和合并复杂度是接近的，于是我们就实现了复杂数据结构的在线化。
- 以这个题为例，时间复杂度就是 $O(\sum |s_i| \log(\sum |s_i|))$ ，空间复杂度精细实现是线性的。

题意

给出一个字符串 s 和 n 个字符串 p_i , 再给出一个常数 k , 求每个字符串 p_i 在 s 中的出现次数。本题中两个字符串 a, b 相等的定义如下:

- $|a| = |b|$
- 对于所有 $a_i \neq b_i$ 且 $a_j \neq b_j$, $|i - j| < k$

$$|s|, \sum |p_i| \leq 2 \times 10^5$$

- 字符串相等的条件等价于 $|a| = |b|$ 且 $lcp(a, b) + lcs(a, b) + k \geq |a|$ 。
- 多串匹配考虑 ACAM，建出正串和反串的 ACAM。
- 对于一个 s 的子串 $s[x, x + |p_i| - 1]$ ，和 p_i 匹配的条件为 s 存在长度不超过 k 的子串 $s[l, r]$ 使得在正串 ACAM 能匹配到 $p_i[1, l - x]$ ，反串 ACAM 能匹配到 $p'_i[1, r - x]$ 。
- 考虑强制让中间间隔为 k ，这样可能会造成计数重复，不过之后减去 $k' = k - 1$ 的答案可以差分出真正的答案。

- 根据 fail 树的性质，每个子树内的节点的代表串的 border 之一是子树根的代表串。
- 设 $p_i[1, j]$ 在正串 ACAM 上的节点为 u ，后缀 $p'_i[1, |p_i| - j - k]$ 在反串 ACAM 上的节点为 v 。
- 设 s 中以 pos 结尾的后缀，在 ACAM 上的节点是 x ，以 $pos + k + 1$ 开始的前缀节点为 y 。
- 则 (x, y) 对 (u, v) 有贡献当且仅当 x 在 u 子树内且 y 在 v 子树内。
- 这是一个二维数点问题，时间复杂度为 $O(n \log n)$ 。

题意

给出 n 个 s 串, m 个 t 串。定义 $f(i, j, k, l)$ 表示 $s_i + t_j$ 作为 $s_k + t_l$ 的子串出现的次数。求

$$\sum_{i=1}^n \sum_{j=1}^m \sum_{k=1}^n \sum_{l=1}^m f(i, j, k, l)$$

$$\sum |s_i|, \sum |t_j| \leq 10^6$$

- 分类讨论。先讨论 $s_i + t_j$ 仅出现在 s_k 的情况，仅出现在 t_l 同理。
- 建出 ACAM，对于每个字符串 s_k 的每个位置，预处理有多少 s_i 作为后缀在这里结尾，以及有多少 t_j 作为前缀在这里开始。
- 利用 ACAM 很容易做到这些。最后枚举每个位置，把这两个信息乘起来累加即可。

- 第二种情况: s_i 是 s_k 的后缀, t_j 是 t_l 的前缀。
- 利用第一种情况预处理的信息也可以解决。

- 第三种情况: s_i 是 s_k 中, t_j 跨过 s_k 和 t_l 。
- 枚举 t_j 的每个位置, 利用哈希和第一种情况预处理的后缀信息即可解决。
- 综上, 复杂度为 $O(\sum |s_i|, \sum |t_j|)$ 。

Q & A



Thank you all!

祝各位学习顺利!

